

Lab Report: Automated Product Listing Generator

Lab: API Calling to ChatGPT

Model: GPT-4o-mini

Implementation: Python / Jupyter Notebook

Author: Sergii Rostovskyi

1. How the API Integration Works

The project implements an automated product listing generator using the OpenAI API and the `gpt-4o-mini` multimodal model.

The workflow consists of seven steps:

1. Initialize the OpenAI client using an API key loaded from an environment variable.
2. Load fashion product data from the Hugging Face dataset `ashraq/fashion-product-images-small` and prepare 100 products.
3. Save product images locally and encode them in Base64.
4. Construct prompts containing the product name, category, price, and additional attributes.
5. Send the text prompt and product image to the OpenAI API.
6. Parse the model's JSON response and save generated listings to `generated_listings.json`.
7. Demonstrate error handling for missing images, authentication failures, and malformed JSON.

Each generated listing contains a product title, description, features, and SEO keywords. The notebook also generates PNG evidence of successful API calls, generated listings, and error handling.

2. Challenges Faced

Image processing: The dataset provides image objects rather than ready-to-use file paths. Images needed to be saved locally and converted to Base64 before being sent through the API.

Missing prices: The original dataset does not contain product prices. Placeholder prices were assigned based on product categories. These prices are illustrative and do not represent actual retail values.

Response formatting: LLM-generated responses may contain Markdown code fences or invalid JSON. A parsing function removes formatting markers before JSON decoding.

Error handling: The implementation uses exception handling to prevent individual failures from interrupting batch processing. Authentication-error handling was demonstrated using a simulated HTTP 401 response.

3. Quality of Generated Listings

The notebook successfully executed its processing and API integration steps. Generated product listings were saved in structured JSON format.

The prompts specify the required content and output structure, helping produce consistent descriptions, feature lists, and SEO keywords.

However, image-based generation has limitations. The model may infer product characteristics that cannot be reliably confirmed from an image. Descriptions should therefore be reviewed against product images and source metadata before publication.

Placeholder prices must also be replaced with actual product data in a production application.

4. Potential Improvements

Several improvements could make the application more reliable and scalable:

- Implement exponential backoff and retry logic for temporary API failures and rate limits.
- Use JSON Schema validation to enforce consistent output structures.
- Process multiple images per product to improve visual understanding.
- Add automated checks for unsupported claims and consistency with product metadata.
- Introduce configurable writing styles for different audiences and marketplaces.
- Track API usage, costs, processing time, and generation quality.
- Support incremental processing and checkpoints for larger product catalogs.

Conclusion

This laboratory demonstrated how to integrate a multimodal LLM into a practical Python workflow. It combined dataset preparation, image encoding, prompt engineering, API calls, structured JSON processing, automation, and error handling.

The resulting solution illustrates how generative AI can reduce repetitive content creation while maintaining a consistent product listing structure. Human review and stronger validation would be essential before production deployment.